

STRUKTURNI PATERNI

ADAPTER PATTERN:

- Adapter patern služi za širu upotrebu već postojećih klasa.
- U našem dijagramu adapter patern najbolje je primijeniti na klasu notifikacija.
- Ovaj patern rješava problem povezanosti sa jednim načinom slanja, uvodeći adaptere za npr. SMS, Email i Push obavijesti.
- Time postizemo da klasa rezervacija ostane nepromijenjena bez obzira na to koji konkretni komunikacijski sistem koristimo u pozadini.

BRIDGE PATTERN:

- Bridge pattern služi za razdvajanje apstrakcije od implementacije kako bi se obje mogle nezavisno mijenjati.
- U našem dijagramu bridge pattern najbolje je primijeniti na sistem notifikacija.
- Ovaj pattern rješava problem vezivanja tipa notifikacije za konkretan način slanja, uvodeći apstrakciju (notifikacija) i implementaciju (slanjePoruke).
- Na taj način možemo imati različite tipove notifikacija (npr. podsjetnik, potvrda rezervacije) i različite načine slanja (SMS, Email, Push), koji se mogu kombinovati bez potrebe za kreiranjem velikog broja klasa za svaku kombinaciju.
- Time postizemo fleksibilnost sistema i lakše proširenje bez izmjena postojećeg koda.

COMPOSITE PATTERN

- Composite pattern služi za organizovanje objekata u strukturu stabla, tako da se pojedinačni objekti i grupe objekata mogu tretirati na isti način.
- U našem dijagramu composite pattern najbolje je primijeniti na sistem rezervacija ili izvještaja.
- Ovaj pattern rješava problem rada sa pojedinačnim izvještajima i grupama izvještaja (npr. dnevni, sedmični izvještaji), uvodeći zajednički interfejs za obje vrste objekata.
- Na taj način možemo imati jednostavne elemente (pojedinačni izvještaj) i složene elemente (kolekcija izvještaja), nad kojima se izvršavaju iste operacije.
- Time postizemo jednostavniji rad sa hijerarhijskim strukturama, bez potrebe za razlikovanjem da li radimo sa jednim ili više objekata.

FACADE PATTERN

-Facade pattern služi za pojednostavljenje korištenja kompleksnog sistema tako što pruža jedinstven, pojednostavljen interfejs prema klijentu.

-U našem dijagramu facade pattern najbolje je primijeniti na sistem rezervacija.

-Ovaj pattern rješava problem direktne komunikacije klijenta sa više različitih klasa (npr. rezervacija, notifikacije, popusti), uvodeći jednu centralnu klasu koja objedinjuje sve potrebne operacije.

-Na taj način klijent komunicira samo sa fasadom, dok ona u pozadini koordinira rad ostalih komponenti sistema.

-Time postižemo smanjenje složenosti, bolju organizaciju koda i lakše korištenje sistema bez poznavanja njegove strukture.

PROXY PATTERN

-U naš dijagram klasa dodali smo zaštitni proxy pattern

-Na taj način kontrolišemo kontrolu pristupa rezervaciji

-Proxy implementira isti interfejs kao i stvarna klasa rezervacije, ali prije prosljeđivanja zahtjeva vrši dodatne provjere

-Time se omogućava da samo ovlašteni korisnici mogu izvršavati određene operacije nad rezervacijom.

-Ovim pristupom povećavamo sigurnost sistema

-Proxy kontroliše pristup i sprječava bilo kakvu interakciju sa stvarnim objektom dok se ne ispuni uslov autentifikacije

DECORATOR PATTERN

-U našem dijagramu ovaj pattern je primijenjen na sistem popusta, gdje se osnovna cijena rezervacije može nadograditi različitim pogodnostima.

-Ovaj pattern rješava problem krutog nasljeđivanja, omogućavajući nam da kombinujemo više različitih popusta (npr. sezonski i popust za rani termin) na jednoj rezervaciji.

-Na taj način postižemo da klasa rezervacija ostane jednostavna, dok se logika izračunavanja složenih popusta nalazi u posebnim dekorator klasama.

-Time dobijamo sistem koji je lako proširiv, jer novi tipovi popusta ne zahtijevaju prepravljanje postojeće logike naplate ili baze podataka.